

RootTickRunner 设计概览

背景

本仓库的 `RootTickRunner` 旨在解决大型 Unity 项目中常见的模块初始化顺序与 Tick 调度混乱问题。通过将根逻辑与 Tick 调度合并成一个核心脚本，结合数据驱动模块顺序表，可以消除模块之间互相引用导致的启动纠缠，并提供灵活的生命周期管理。下文详细说明了各部件的设计与使用方法。

基础概念

数据表与 `systemInitOrder`

项目使用 [UnityJsonDataEditor](#) 维护 JSON 数据表。运行时使用 `DataEntityRuntimeLoader` 加载数据表，`Initialize()` 必须在读取任何数据前调用 ¹。模块顺序由表格 `systemInitOrder` 描述，其字段含义如下：

字段名	类型	描述
<code>id</code>	int	初始化顺序（越小越先）
<code>name</code>	string	模块名称，需要与实现类的 <code>Name</code> 属性一致
<code>ticktype</code>	int	Tick 调用阶段： <code>0</code> 为 <code>Update</code> ， <code>1</code> 为 <code>LateUpdate</code>

每个需要调度的业务模块都必须在此表中注册，表结构可通过 `DataEntityRuntimeLoader.GetSchema("systemInitOrder")` 获取并遍历 ²。

事件总线

事件总线 `EventBus` 负责在系统内发布和订阅事件。初始化时通过 `Init(IDataTableRuntime)` 传入表运行时对象，它会根据配置从数据表加载可用的事件和标签。业务模块在 `Init()` 中可通过 `SubscribeEvent()` 或 `SubscribeTag()` 订阅事件，在适当时机调用 `PublishEvent()` 发布事件。

RootTickRunner 责任与流程

核心职责

- 单例管理：**`RootTickRunner` 作为全局唯一实例，使用 `DontDestroyOnLoad` 确保跨场景存在。其他脚本通过 `RootTickRunner.Instance` 获取引用。
- 基础模块初始化：**在 `Awake()` 中调用 `DataEntityRuntimeLoader.Initialize()` 加载数据表，再获取或添加 `EventBus` 组件并通过 `eventBus.Init(DataEntityRuntimeLoader)` 初始化事件表和标签。
- 读取模块表：**调用 `LoadSystemInitOrder()` 从 `systemInitOrder` 表读取业务模块的初始化顺序、Tick 类型，并构建内部 `Dictionary` 保存。

4. **注册与初始化模块**：提供 `RegisterModule(ISystemModule module)` 接口供业务模块在自身 `Start()` 中注册。只有当表中定义的所有模块都注册完成后，才会按顺序调用各模块的 `Init()`。
5. **调度 Tick**：在 `Update()` 和 `LateUpdate()` 中遍历模块列表，分别执行 `ticktype=0` 和 `ticktype=1` 的模块的 `Tick()`。默认 Tick 顺序与初始化顺序一致，可根据需要扩展更多阶段或单独的 Tick 顺序表。
6. **日志输出**：在基础模块初始化、每个业务模块初始化完成及所有模块初始化完成时分别输出日志，便于调试启动流程。
7. **服务字典与跨模块引用**：`RootTickRunner` 维护一个以字符串为键的服务字典，用于存放基础服务（如数据库与事件总线）及业务模块实例。业务模块可以通过 `GetService<T>(name)` 根据名称获取其他模块或服务的引用，实现系统间解耦调用。

生命周期流程

1. **Awake 阶段**
2. 检查并设置 `RootTickRunner` 单例，调用 `DontDestroyOnLoad`。
3. 调用 `DataEntityRuntimeLoader.Initialize()` 初始化运行时数据表^①。
4. 获取或添加 `EventBus` 组件，配置相关字段后调用 `eventBus.Init(DataEntityRuntimeLoader)`。
5. 调用 `LoadSystemInitOrder()` 读取 `systemInitOrder` 表并构建模块字典。
6. 打印基础模块初始化完成日志。
7. **模块注册阶段**
8. 每个业务模块实现 `ISystemModule` 接口，在其 `Start()` 中调用 `RootTickRunner.Instance.RegisterModule(this)` 注册。
9. `RegisterModule()` 会将模块与表中定义匹配并记录注册状态；当发现所有模块都已注册完成时，立即触发 `InitAllModules()`。
10. **服务注入**：注册时 `RootTickRunner` 会自动将该模块以其名称加入服务字典。基础服务（数据库和事件总线）也会在初始化时注入。例如，可以通过 `RootTickRunner.Instance.GetService<EventBus>("eventbus")` 获取事件总线实例，通过 `GetService<BattleSystem>("battlesystem")` 获取其它业务模块实例。
11. **Init 阶段**
12. `InitAllModules()` 根据表中 `id` 顺序调用每个模块的 `Init(RootTickRunner root, EventBus bus)` 方法。
13. 每个模块初始化成功后打印日志 `[RootTickRunner] 模块 '<Name>' 初始化完成`。
14. 最后打印 `[RootTickRunner] 所有业务模块初始化完成` 标志整个初始化流程结束。
15. **Tick 阶段**
16. 在 `Update()` 中遍历模块，按顺序调用 `ticktype=0` 的模块的 `Tick()`；在 `LateUpdate()` 中调用 `ticktype=1` 的模块的 `Tick()`。
17. 如果需要更多阶段，可以扩展数据表中 `ticktype` 值并在 `Update` / `LateUpdate` 之外调用。

业务模块实现

业务模块需实现 `ISystemModule` 接口，示例：

```
using GameFramework;
using UnityEngine;

public class BattleSystem : MonoBehaviour, ISystemModule
{
    public string Name => "battlesystem";

    public void Init(RootTickRunner root, EventBus bus)
    {
        // 在此注册事件、加载资源、初始化状态等
        bus.SubscribeEvent(this, "OnBattleStart", OnBattleStart);
    }

    public void Tick()
    {
        // 每帧更新逻辑
    }

    private void OnBattleStart(object payload)
    {
        // 事件回调
    }

    void Start()
    {
        // 注册自己，等待 Root 调用 Init()
        // 在调用注册之前获取 RootTickRunner 单例
        var root = RootTickRunner.Instance;
        root.RegisterModule(this);
    }
}
```

注意事项：

- 模块名称必须与 `systemInitOrder` 表中的 `name` 字段完全一致，否则无法注册成功。
- 请不要在 `Awake()` 或 `Start()` 中直接执行初始化或 `Tick` 逻辑，应当在 `Init()` 中进行初始化，在 `Tick()` 中更新。
- 若模块需要在特定阶段运行非帧循环逻辑，可以在 `Init()` 中通过 `EventBus` 订阅事件并在回调中处理。
- 当业务模块需要访问其他模块或基础服务时，先通过 `RootTickRunner.Instance` 获取根引用，再调用 `GetService<T>("服务名")` 获得对应实例。例如：

```

var root    = RootTickRunner.Instance;
var audio   = root.GetService<AudioSystem>("audiosystem");
var bus     = root.GetService<EventBus>("eventbus");
if (audio != null)
{
    // 调用其他模块的方法
    audio.PlayBackgroundMusic();
}

```

若返回 null，则说明该服务尚未注册或名称不匹配。

扩展建议

- **异步加载支持**：目前设计假定 `DataEntityRuntimeLoader.Initialize()` 和表读取为同步操作；如需支持异步，请使用协程或 `async/await` 包装初始化过程，并适当延迟模块初始化。
- **Tick 阶段扩展**：可新增字段 `tickPhase` 表示自定义更新阶段，例如固定时间步、场景加载后才执行等；在 Root 中增加对应的调用逻辑。
- **模块依赖检查**：在 `LoadSystemInitOrder()` 时记录各模块依赖关系，注册时校验依赖是否存在并在逻辑上禁止依赖倒置。

结语

`RootTickRunner` 提供了一套可靠的初始化与 Tick 调度框架，结合数据驱动模块顺序表可以轻松管理多个系统模块的生命周期，避免了模块之间相互引用引起的复杂依赖。通过日志可以清晰地观察初始化过程，对调试大型项目大有裨益。如在使用中遇到问题或需要进一步扩展，欢迎反馈并讨论改进方案。

1 2 [GitHub - Duckkings/UnityJsonDataEditor: 数据表编辑器](#)

<https://github.com/Duckkings/UnityJsonDataEditor>